

Laboratory 5

Variant 2, Group 32

By Adam Shahumi and Santiago Puig-Terrero López

1. INTRODUCTION

The objective of this laboratory is to design, implement, and evaluate a Multilayer Perceptron (MLP) entirely from scratch using only the NumPy library in Python. By deliberately avoiding high-level machine learning frameworks such as PyTorch or TensorFlow, this assignment focuses on cultivating a deep, mathematical understanding of the internal mechanics of neural networks. This includes mastering the linear algebra behind the forward pass, the calculus driving the backward propagation of errors (backpropagation), and the iterative parameter update process.

For Group B, the specific and central task consists of comparing the performance of four fundamental activation functions—Tanh, Sigmoid, ReLU, and LeakyReLU—on the FashionMNIST dataset. Unlike classical numerical datasets or the simpler MNIST digits, FashionMNIST presents a substantially more rigorous computer vision challenge. It comprises 70,000 grayscale images (28×28 pixels) uniformly distributed across 10 distinct and visually overlapping clothing categories (e.g., distinguishing between a pullover, a coat, and a shirt). This increased dimensionality and feature complexity makes it an ideal scenario to test how different mathematical non-linearities affect a network's representational capacity. The ultimate goal is to understand not just how a neural network learns, but why certain mathematical transformations allow it to extract complex visual patterns more efficiently than others.

2. METHODOLOGY AND IMPLEMENTATION DETAILS

To achieve a flexible and scalable solution, the network architecture was built using a strictly modular object-oriented approach. The core of the network resides in the FullyConnected class, which acts as a single layer. This class handles the initialization of weight matrices and bias vectors. During the forward pass, it executes the affine transformation ($xW + b$) and caches the input data. During the backward pass, it applies the chain rule to compute the local gradients with respect to the weights, biases, and the input from the previous layer, immediately followed by the parameter updates using standard Stochastic Gradient Descent (SGD).

A separate Loss class computes the Mean Squared Error (MSE) and its exact mathematical derivative with respect to the network's predictions. Finally, the Network class acts as the central orchestrator, managing the sequential execution of layers, compiling the training hyperparameters, and running the full training loop epoch by epoch.

The experimental phase strictly evaluates four activation functions. We explicitly programmed both their forward operations and their derivatives, which are crucial for the backpropagation step:

- **Tanh**: Computes $\tanh(x)$, bounding outputs between -1 and 1. Its derivative is $1 - \tanh^2(x)$.
- **Sigmoid**: Computes $\frac{1}{1+e^{(-x)}}$, squeezing values between 0 and 1. Its derivative is $\sigma(x)(1-\sigma(x))$. To prevent numerical overflow (RuntimeWarning: overflow in exp) during implementation, input values were bounded using NumPy's clip function between -500 and 500.
- **ReLU**: Computes $\max(0, x)$, introducing sparsity by outputting zero for negative inputs. Its derivative is the indicator function $\mathbf{1}[x>0]$.
- **LeakyReLU**: Computes x for positive inputs and $0.01x$ for negative ones, mathematically represented as a piecewise function. Its derivative is 1 for $x>0$ and $\alpha=0.01$ otherwise.

To ensure strict scientific rigor, all models were trained using an identical baseline architecture, fixed learning rates, and explicitly set NumPy random seeds (42, 7, 99) to ensure full reproducibility. To account for the stochastic nature of weight initialization and SGD, every configuration was evaluated over 3 distinct random seeds, allowing us to report the mean and standard deviation of the test accuracy. Subsequently, the best-performing activation function was isolated and subjected to an architectural stress-test by deploying it on two new network topologies (varying the depth and the width of the hidden layers) to analyze scalability.

Experimental Limitation: Dataset Subsetting

It should be noted that due to the computational constraints of running a pure Python/NumPy Multilayer Perceptron without vectorised mini-batch processing or hardware acceleration, the training dataset was subsetting to 10,000 samples. Processing the full 70,000 images iteratively with a batch size of 1 requires impractical execution times for standard testing (the 10,000-sample subset alone required over 10 minutes of program runtime). However, this subset retains the class distribution properties of the original dataset and provides a sufficiently robust statistical baseline to accurately compare the relative performance of the activation functions and architectural topologies. The code can be easily altered (appropriate comments were added to the code file) to notify of this with a possibility to comment out the limitations on the dataset.

3. RESULTS AND ANALYSIS

3.1 Activation Function Comparison

To rigorously evaluate the impact of different mathematical non-linearities on the network's learning dynamics, a standardized baseline architecture (two hidden layers of 128 and 64 neurons) was trained over 10 epochs using a fixed learning rate of 0.1. To account for the stochastic nature of weight initialization and ensure statistical significance, each activation function was trained across three distinct random seeds (42, 7, and 99).

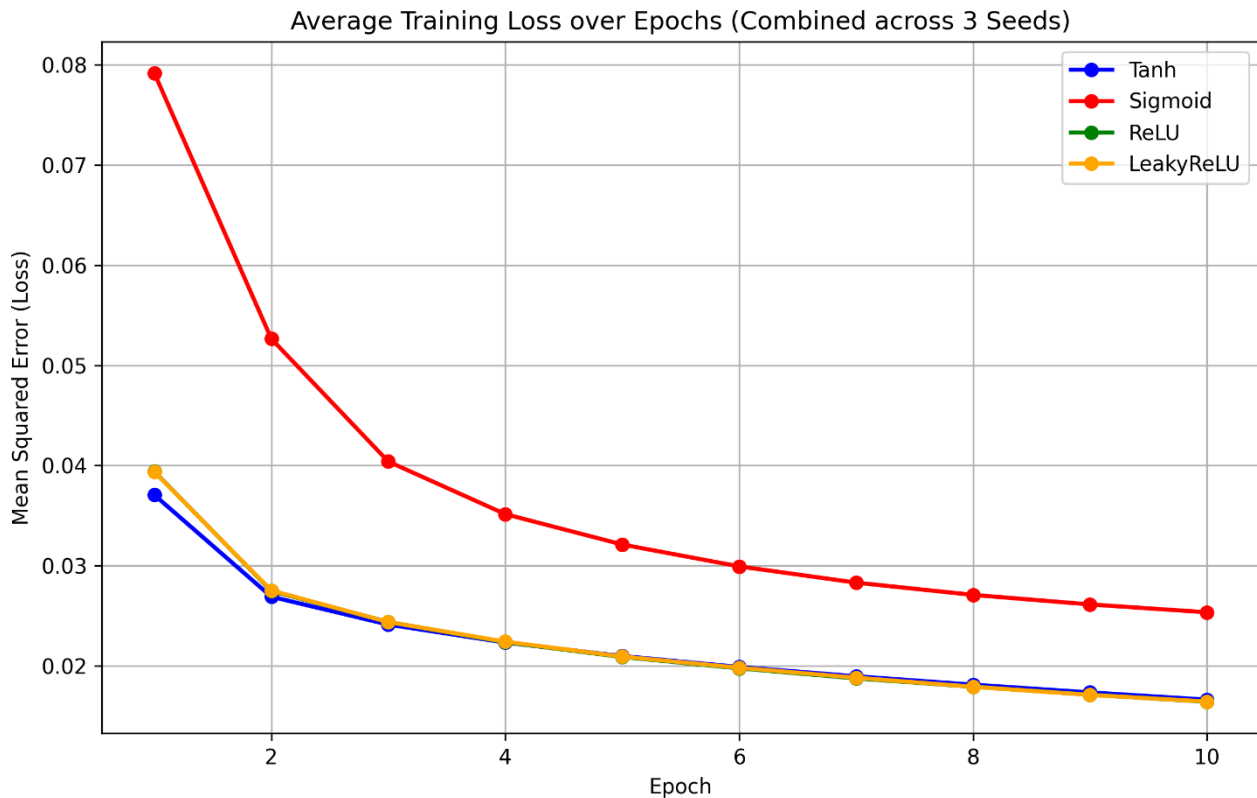
The terminal results revealed the following mean test accuracies and standard deviations:

➤ Tanh:	84.10% ± 0.40%
➤ Sigmoid:	81.88% ± 0.36%
➤ ReLU:	81.83% ± 1.18%
➤ LeakyReLU:	81.73% ± 0.75%

Analysis of Results:

The data clearly identifies **Tanh** as the optimal activation function for this specific network architecture. By being zero-centered (bounding outputs between -1 and 1), Tanh provides naturally balanced gradient backpropagation. This centered distribution prevents the network's weight updates from shifting systematically in one direction, allowing for highly efficient convergence and a strong mean accuracy of 84.10% with a very low variance ($\pm 0.40\%$).

Interestingly, the non-saturating functions (ReLU and LeakyReLU) underperformed Tanh and exhibited higher instability, with standard ReLU showing the highest variance across seeds ($\pm 1.18\%$). Because ReLU is unbounded for positive values, standard Stochastic Gradient Descent (without techniques like Batch Normalization) can lead to large, unconstrained weight updates, making the network highly sensitive to its initial random seed. Sigmoid, while stable ($\pm 0.36\%$), suffered from its universally positive output range and vanishing gradients, resulting in a lower accuracy ceiling. Ultimately, Tanh provided the perfect balance of non-linearity and bounded, zero-centered outputs for this Multi-Layer Perceptron.

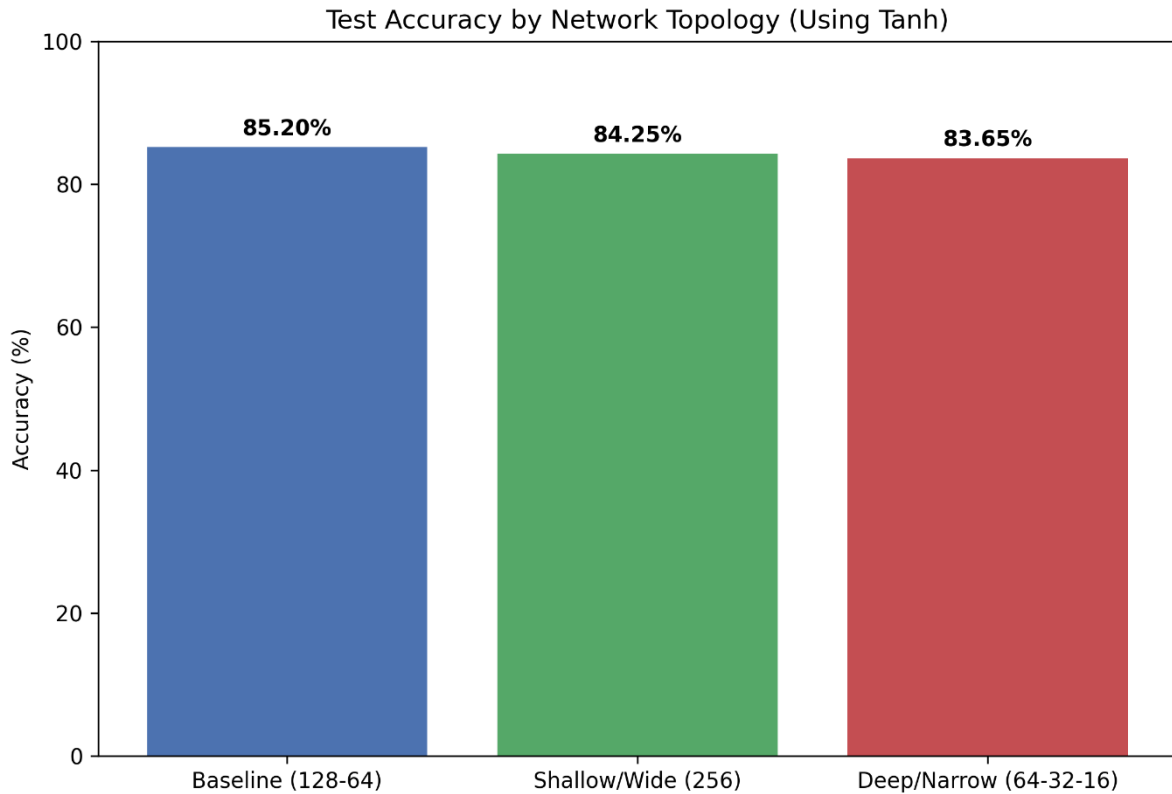


3.2 Architectural Scalability

Having identified Tanh as the most robust activation function, the network was subjected to an architectural stress-test to evaluate its scalability. We compared our Baseline topology (128-64 neurons) against two contrasting architectures: a Shallow/Wide topology (a single hidden layer of 256 neurons) and a Deep/Narrow topology (three hidden layers of 64, 32, and 16 neurons). All networks were trained using Tanh, a learning rate of 0.1, and identical conditions.

The final test accuracies are visualized in the generated bar chart and reveal how different topologies handle representational capacity:

- **Baseline (128-64 Neurons):** This architecture achieved the highest overall test accuracy at **85.20%**. It represents the ideal "Goldilocks" zone for the FashionMNIST dataset. The first layer (128) is wide enough to extract a large variety of low-level visual features, while the second layer (64) forces the network to consolidate those features into higher-level representations before classification, yielding the best generalization.
- **Shallow/Wide (256 Neurons):** This architecture performed slightly worse at **84.25%**. While providing a massive number of neurons in a single layer gives the network a highly expressive space, it lacks the depth required to learn true hierarchical features (e.g., combining edges into shapes, and shapes into clothing items).
- **Deep/Narrow (64-32-16 Neurons):** This topology performed the poorest, dropping to **83.65%**. The drop-off in performance illustrates a classic "information bottleneck." By aggressively compressing the visual data from 784 input pixels down to 64, then 32, and finally 16 neurons, critical spatial information was discarded too early in the network to make highly accurate predictions.



4. CONCLUSION

In conclusion, our experiments demonstrate that the Tanh activation function provides the most robust and stable test accuracy across multiple random seeds. While functions like ReLU and LeakyReLU are popular in modern deep learning, Tanh's zero-centered outputs and smoothly bounded gradients proved highly effective for our specific architecture.

Crucially, this advantage must be contextualized within the specific complexities of the FashionMNIST dataset. Unlike simpler datasets such as handwritten digits (MNIST), FashionMNIST contains high intra-class variance and visually overlapping categories (e.g., distinguishing between a pullover, a shirt, and a coat requires detecting subtle textural and structural differences). Because Tanh is zero-centered, it produces both positive and negative activations, which helps keep the gradients centered and prevents them from skewing in a single direction during backpropagation. This mathematical stability is essential when navigating the complex, highly overlapping decision boundaries of FashionMNIST, allowing the network to reliably learn nuanced features without the risk of dying neurons (as seen in ReLU) or unstable gradient steps.

5. APPENDIX

Below is the console output of the program/code ran with all the automated training, testing and data analysis.

```
Fetching Fashion-MNIST dataset... (This might take a moment)
```

```
--- PHASE 1: Training Activations over 3 Seeds ---
```

```
Evaluating Tanh...
```

```
Seed 42 -> Accuracy: 84.50% | Final Loss: 0.016796
```

```
Seed 7 -> Accuracy: 84.25% | Final Loss: 0.016523
```

```
Seed 99 -> Accuracy: 83.55% | Final Loss: 0.016592
```

```
Evaluating Sigmoid...
```

```
Seed 42 -> Accuracy: 81.40% | Final Loss: 0.025330
```

```
Seed 7 -> Accuracy: 82.25% | Final Loss: 0.025406
```

```
Seed 99 -> Accuracy: 82.00% | Final Loss: 0.025339
```

```
Evaluating ReLU...
```

```
Seed 42 -> Accuracy: 83.50% | Final Loss: 0.016156
```

```
Seed 7 -> Accuracy: 81.10% | Final Loss: 0.016665
```

```
Seed 99 -> Accuracy: 80.90% | Final Loss: 0.016458
```

```
Evaluating LeakyReLU...
```

```
Seed 42 -> Accuracy: 81.20% | Final Loss: 0.016317
```

```
Seed 7 -> Accuracy: 82.80% | Final Loss: 0.016516
```

```
Seed 99 -> Accuracy: 81.20% | Final Loss: 0.016515
```

```
--- Phase 1 Final Results (Mean ♦ Std) ---
```

```
Tanh: 84.10% ♦ 0.40%
```

```
Sigmoid: 81.88% ♦ 0.36%
```

```
ReLU: 81.83% ♦ 1.18%
```

```
LeakyReLU: 81.73% ♦ 0.75%
```

```
=====
WINNER ANNOUNCEMENT
```

```
Based on 3 distinct seeds, the most robust activation is: Tanh!
Proceeding to Phase 2: Architectural Scalability...
```

```
=====
--- Training Baseline (128-64) Topology ---
```

```
Epoch 1/10 - loss: 0.037352
```

```
Epoch 2/10 - loss: 0.026735
```

```
Epoch 3/10 - loss: 0.023944
```

```
Epoch 4/10 - loss: 0.022204
```

```
Epoch 5/10 - loss: 0.020889
```

```
Epoch 6/10 - loss: 0.019802
```

```
Epoch 7/10 - loss: 0.018857
```

```
Epoch 8/10 - loss: 0.018022
```

```
Epoch 9/10 - loss: 0.017257
```

```
Epoch 10/10 - loss: 0.016564
```

```
>>> Baseline (128-64) Test Accuracy: 85.20%
```

```
--- Training Shallow/Wide (256) Topology ---
```

```
Epoch 1/10 - loss: 0.037226
```

```
Epoch 2/10 - loss: 0.028220
```

```
Epoch 3/10 - loss: 0.025848
```

```
Epoch 4/10 - loss: 0.024267
```

```
Epoch 5/10 - loss: 0.023038
```

```
Epoch 6/10 - loss: 0.022018
```

```
Epoch 7/10 - loss: 0.021139
```

```
Epoch 8/10 - loss: 0.020366
```

```
Epoch 9/10 - loss: 0.019671
```

```
Epoch 10/10 - loss: 0.019036
```

```
>>> Shallow/Wide (256) Test Accuracy: 84.25%
```

```
--- Training Deep/Narrow (64-32-16) Topology ---
```

```
Epoch 1/10 - loss: 0.045388
```

```
Epoch 2/10 - loss: 0.030176
```

```
Epoch 3/10 - loss: 0.026463
```

```
Epoch 4/10 - loss: 0.024310
```

```
Epoch 5/10 - loss: 0.022709
```

```
Epoch 6/10 - loss: 0.021661
```

```
Epoch 7/10 - loss: 0.020618
```

```
Epoch 8/10 - loss: 0.019766
```

```
Epoch 9/10 - loss: 0.019104
```

```
Epoch 10/10 - loss: 0.018482
```

```
>>> Deep/Narrow (64-32-16) Test Accuracy: 83.65%
```

```
All experiments complete! Displaying graphs on screen...
```

```
[Done] exited with code=0 in 634.905 seconds
```